CoinMarketAnalyzer

Self-Learning Intelligence Layer

System-Wide Optimization + Per-User Adaptive Profiles

1. Executive Summary

We are building a two-layer self-learning intelligence system:

Layer 1 System-Wide Intelligence: Analyzes aggregated prediction outcomes across ALL users to optimize the core prediction engine. This makes the base predictions better for everyone by automatically tuning layer weights, buy/sell thresholds, and timing parameters based on proven accuracy data.

Layer 2 Per-User Adaptive Profiles: Automatically builds a behavioral profile for each user based purely on their usage patterns. Over time, the system learns each user's trading style, tracks their personal accuracy, identifies their recurring mistakes, and personalizes confidence scores, timing advice, and risk warnings specifically for them.

Both layers use an LLM (Claude or GPT-4o) as the reasoning engine, not hardcoded rules. The LLM analyzes data and proposes changes; the system validates, tests, and applies them safely.

1.3 Key Principles

- Non-destructive the existing prediction pipeline remains unchanged; intelligence layers add on top
- Fully automatic user profiles built from usage data only, no manual configuration required
- LLM-powered reasoning not rule-based; the LLM analyzes patterns and proposes data-driven adjustments
- Safe by design all changes are bounded, shadow-tested, and auto-rolled-back if they hurt performance
- Cost-efficient LLM runs in batch mode (not per-request), keeping API response times unchanged
- Privacy-conscious per-user profiles are isolated and never shared across users

2. Current System Overview

The existing CoinMarketAnalyzer processes prediction requests through a deterministic multi-stage pipeline:

#	Stage	Description

1	**Data Ingestion**	Fetches market candles, holder data, whale activity, and global market context (fear/greed, regime) from Birdeye, CryptoRank, and internal APIs.
2	**Feature Engineering**	TechnicalIndicatorEngine computes RSI, MACD, Bollinger Bands, etc. Recent 24h analysis extracts buy/sell pressure, volume trends, and momentum.
3	**Multi-Layer Scoring**	Five independent layers each produce a signal: ML Model (XGBoost), Whale Engine, Technical Analysis, Holder Metrics, and User Profile/Risk. Each outputs direction + confidence.
4	**Layer Aggregation**	LayerAggregator fuses the five signals using weighted averages. Weights are hardcoded in settings.py (LayerWeightConfig). Agreement bonuses and coverage penalties applied.
5	**Safety Overrides**	Hard safety rules check for rug-pull indicators, high concentration, and extreme market conditions. Can force-override any signal to SELL or HOLD.
6	**Timing & Risk**	EntryTimingEngine determines ENTER_NOW / WAIT / EXIT. Risk management computes stop-loss, take-profit, position sizing.
7	**Response**	Signal, confidence, layer breakdown, timing, risk parameters, and narrative summary assembled and returned.

THE GAP
There is zero feedback from predictions back into the system. The pipeline is entirely one-directional. Layer weights never adjust. Thresholds never change. The system treats all users identically. It cannot distinguish between a configuration producing 40% accuracy vs 70% because it never checks.

3. Two-Layer Intelligence Architecture

The intelligence system operates on two distinct levels that work together but serve different purposes:

	System-Wide (Layer 1)	**Per-User (Layer 2)**	**Combined Effect**
Scope	All users equally	Each user individually	Better base + personal tuning
What it optimizes	Layer weights, buy/sell thresholds, timing bands	Confidence adjustment, timing style, personalized warnings	Core engine accuracy + individual relevance
Data source	Aggregated outcomes across all users	Individual user's prediction history + outcomes	Macro patterns + micro patterns
Update frequency	Weekly	Weekly (active users) or on-demand	Continuous improvement cycle
LLM calls	1–2 per week	1 per user per update cycle	Batch mode, never per-request
Where it acts	Inside LayerAggregator (pre-signal)	Post-signal, before response (post-processing)	Pipeline + personalization

Think of it this way: Layer 1 makes the chef cook better food for everyone. Layer 2 adjusts the seasoning to each diner's taste.

4. Layer 1 System-Wide Intelligence

This layer closes the feedback loop between predictions and the core engine. It answers: "Are our layer weights and thresholds optimal, and how should they change based on actual outcomes?"

4.1 How It Works

1. Every prediction is logged to PostgreSQL with full context: signal, confidence, layer breakdown, market context, safety overrides applied, timing recommendation.
2. Every 4 hours, a background job fetches actual prices for recent predictions and records whether the direction was correct at 4h, 12h, and 24h timeframes.
3. Weekly, aggregated stats are computed via SQL: per-layer accuracy rates, conflict resolution outcomes (when layers disagreed, who was right?), safety override effectiveness, threshold performance.
4. These stats are sent to the LLM in a single structured prompt. The LLM returns a JSON policy patch with proposed weight adjustments, threshold changes, and reasoning.
5. The proposed policy enters shadow mode: both old and new weights are computed on every prediction, both results logged, but only old weights affect the user-facing response.
6. After sufficient shadow data (500+ predictions with outcomes), metrics are compared. If the new policy outperforms baseline, it's promoted to active. If not, it's discarded.
7. A rolling monitor checks active policy performance daily. If accuracy drops below baseline minus a configurable epsilon, automatic rollback triggers.

4.2 What the LLM Optimizes

Component	What LLM Adjusts	Constraints
Layer Weights	Relative importance of ML, whale, technical, holder, and user_profile layers in LayerAggregator.	Must sum to 1.0. No weight below 0.10 or above 0.45. Changes bounded to ±0.08 per cycle.
Buy/Sell Thresholds	Score cutoffs in _score_to_signal that determine BUY vs HOLD vs SELL. Taper zone boundaries.	Changes capped at ±0.1 from current values per policy version.
ML Calibration	XGBoost probability thresholds (PERPS_LONG_THRESHOLD, etc.). When to trigger retraining.	Thresholds within safe ranges. Retraining is suggested, not auto-triggered.
Timing Bands	WAIT/ENTER_NOW threshold boundaries in TimingConfig.	Changes within ±X% caps. Safety coercions

		(HOLD+ENTER_NOW→WAIT) are immutable.
Safety Overrides	NOTHING in v1. LLM can propose NEW checks (human-approved). Can never weaken existing rules.	All rug-pull, concentration, and regime guards are permanently immutable.

4.3 Policy Lifecycle

Every system policy moves through a strict lifecycle: Proposed → Shadow → Active (or Discarded). No policy can skip shadow testing. Automatic rollback if active policy degrades performance.

5. Layer 2 Per-User Adaptive Profiles

This layer personalizes predictions for each individual user. Profiles are built entirely from usage data with no manual configuration required.

5.1 What the Profile Learns

Request Patterns (Immediate): Which tokens the user checks and how often, token type preferences (meme vs perps ratio), time-of-day activity patterns, market condition preferences (checks more during fear or greed?), request frequency spikes (potential FOMO or panic indicators).

Trading Outcomes (Over Time): Which signals the user followed vs ignored, price movement after each prediction relative to their timeframe, effective hold duration, profit/loss patterns per token type and market condition, which signal layers correlated best with this user's profitable trades.

Behavioral Traits (LLM-Derived): Periodically, the LLM analyzes accumulated data and classifies behavioral traits as continuous scores (0–1), not binary labels:

Trait	Detection Method	Impact on Predictions
Panic Seller	Frequent requests during drops; sells at bottom; high velocity during fear spikes.	Suggests WAIT more often during volatility; highlights historical recovery data; wider stop-loss recommendations.
Peak Buyer	Requests predictions after sharp price increases; buys into late momentum; higher loss rate on high-confidence BUY signals.	Adds caution when signal matches pattern; lowers effective confidence to reduce over-conviction on run-ups.
Diamond Hands	Rarely acts on SELL; holds through drawdowns; long average position duration.	Emphasizes long-term trend data; timing focuses on macro entries; short-term noise de-prioritized.
Scalper	Very frequent requests for same token; acts quickly; short hold times.	Prioritizes short-timeframe indicators; timing precision over directional conviction.

FOMO Trader	Checks new trending tokens never looked at before; spikes correlate with social media volume.	Flags unfamiliarity; extra caution on first-time token checks during hype cycles.
Disciplined	Consistent patterns; follows signals with good hit rate; balanced risk behavior.	Standard predictions without behavioral corrections; system trusts user's judgment.

5.2 How the Profile Improves Predictions

The profile affects three specific areas of the prediction output. It does NOT change the core signal, it adjusts the final response based on what the system has learned about this specific user.

Confidence Adjustment: The displayed confidence score is adjusted based on the user's historical accuracy with similar setups. Example: Base system says BUY with 75% confidence on a meme token during a fear market. But this user has historically lost money 80% of the time in meme+fear+BUY setups. Profile reduces effective confidence to 55% and adds context: "Your past trades in similar conditions had a 20% success rate. Consider waiting for stabilization."

Personalized Timing: Entry/exit recommendations adjust based on trading speed and style. Scalpers get tighter timing windows focused on immediate momentum. Holders get relaxed timing where minor dips don't trigger WAIT. Panic sellers get extended WAIT recommendations and wider stop-losses.

Personalized Risk Warnings: Instead of generic disclaimers, warnings reference the user's actual history. Examples: "This token is up 40% in 4h. In your last 8 similar trades, 6 resulted in losses." or "You've held through 3 recent SELL signals on similar tokens and averaged -15%." These are LLM-generated from real data, not templates.

5.3 Profile Lifecycle

Stage	Predictions	System Behavior	User Experience
Cold Start	0–19	Requests logged, no profile exists. The system operates identically to today.	Standard predictions. No personalization.
Initial	20–49	LLM creates the first profile with preliminary traits and basic accuracy bucketing. Conservative adjustments only.	Slight confidence adjustments. First warnings may appear if clear patterns exist.
Learning	50–199	Weekly updates. Traits stabilize. Accuracy buckets become statistically meaningful.	Noticeable personalization. Timing adjustments active. Specific, relevant warnings.
Mature	200+	Updates continue but changes are smaller. Strong behavioral model. Well-calibrated adjustments.	Full personalization. Predictions tailored to trading style and accuracy history.

6. Technology Stack

Component	Technology	Why This Choice
Activity Logging	PostgreSQL (async writes)	Handles 200K+ writes/day. Complex aggregation queries for both system-wide and per-user analytics. Proven at scale.
Outcome Tracking	Celery Beat + DataFetcher	Scheduled job fetches actual prices at 4h/12h/24h after each prediction. Reuses existing data fetching infrastructure.
Profile Storage	PostgreSQL (JSONB)	User profiles as structured JSONB. Fast reads. Versioned for rollback. JSONB enables flexible schema evolution.
Profile Cache	Redis	Cached user profiles for hot-path reads (~1ms lookup). Invalidated on profile update. 20K profiles ≈ 50–100 MB.
LLM Reasoning	Claude API (primary) / GPT-4o (fallback)	Mirrors existing LLMAnalyzerService + CircuitBreaker pattern. Structured JSON output. Fallback ensures availability.
Async Queue	FastAPI BackgroundTasks → Redis+Celery	BackgroundTasks for Phase 1. Redis queue at scale for guaranteed delivery and batch inserts.
System Policy Store	PostgreSQL (policies table)	Versioned rows with lifecycle status tracking. Shadow metrics and baseline comparison. Full audit trail.
Shadow Testing	Dual-compute in LayerAggregator	Run old+new weights on every prediction, log both. Zero user impact during testing.
ORM / DB Layer	SQLAlchemy Async	Non-blocking database access from FastAPI. Connection pooling for 10–20K concurrent users.

7. Database Design

Six tables handle the complete intelligence system. The existing codebase and tables remain untouched.

7.1 user_predictions

Written on every prediction request. The raw activity feed per user.

Column	Type	Purpose
id	UUID (PK)	Unique prediction record
user_id	VARCHAR (indexed)	Wallet address or API key identifying the user
token / token_type	VARCHAR	Token symbol and type (meme/perps)

signal / confidence	VARCHAR / FLOAT	BUY/SELL/HOLD and base confidence score
adjusted_confidence	FLOAT (nullable)	Confidence after profile adjustment (null if no profile)
layer_breakdown	JSONB	Per-layer signal, confidence, weight snapshot
safety_overrides	JSONB	Which safety rules fired
market_context	JSONB	Fear/greed, regime, BTC dominance snapshot
timing	VARCHAR	ENTER_NOW / WAIT / EXIT recommendation
policy_version	VARCHAR (nullable)	Active system policy version (null = defaults)
profile_version	INT (nullable)	User profile version applied (null = no profile)
created_at	TIMESTAMP (indexed)	Request timestamp

7.2 prediction_outcomes

Filled by background job 4–24h after each prediction. Links actual price movements to predictions.

Column	Type	Purpose
id	UUID (PK)	Unique outcome record
prediction_id	UUID (FK)	Links to the prediction being evaluated
price_at_prediction	FLOAT	Token price at prediction time
price_at_4h / 12h / 24h	FLOAT	Actual prices at checkpoint timeframes
direction_correct_4h/12h/24h	BOOLEAN	Was direction correct at each timeframe?
pnl_pct_24h	FLOAT	Percentage gain/loss at 24h

7.3 user_profiles

One row per user. Updated periodically by the LLM Profile Builder. This is the per-user intelligence brain.

Column	Type	Purpose
user_id	VARCHAR (PK)	Unique user identifier

version	INT	Profile version (increments on each LLM update)
behavioral_traits	JSONB	Trait scores: {panic_seller: 0.7, peak_buyer: 0.3, ...}
accuracy_by_setup	JSONB	Hit rates bucketed by token_type + market_condition + signal
timing_profile	JSONB	Hold duration, preferred timeframe, speed classification
mistake_patterns	JSONB	Recurring mistakes with frequency and recency
confidence_adjustments	JSONB	Per-setup confidence multipliers applied as post-processing
active_warnings	JSONB	LLM-generated personalized warning templates
total_predictions	INT	Count (gates activation: min 20 required)
llm_reasoning	TEXT	LLM's explanation of profile assessment (auditable)
updated_at	TIMESTAMP	Last profile update time

7.4 profile_history

Archives every profile version for rollback and audit trail.

Column	Type	Purpose
id	SERIAL (PK)	Auto-increment
user_id	VARCHAR (FK)	Which user's profile
version	INT	Profile version number
profile_snapshot	JSONB	Complete profile data at this version
created_at	TIMESTAMP	When this version was created

7.5 system_policies

Stores system-wide optimization policies with full lifecycle tracking.

Column	Type	Purpose
id	SERIAL (PK)	Policy ID
version	VARCHAR	v001, v002, etc.
weights / thresholds	JSONB	Layer weight and threshold adjustments

status	VARCHAR	proposed → shadow → active → rolled_back
shadow_metrics / baseline_metrics	JSONB	Performance comparison data
reasoning	TEXT	LLM's explanation of proposed changes
llm_model_used	VARCHAR	Which LLM generated this policy
promoted_at / rolled_back_at	TIMESTAMP	Lifecycle timestamps

7.6 shadow_predictions

Stores shadow-mode dual-compute results for system policy A/B testing.

Column	Type	Purpose
prediction_id	UUID (FK)	Links to the original prediction
policy_version	VARCHAR	Which proposed policy was shadow-tested
shadow_signal / shadow_confidence	VARCHAR / FLOAT	What the proposed policy would have produced
baseline_signal / baseline_confidence	VARCHAR / FLOAT	What the current active policy produced

8. Complete Pipeline Flow

8.1 Per-Request (Hot Path)

1. Request arrives with user_id (wallet/API key) + token.
2. Existing pipeline runs unchanged: data fetch → indicators → 5 layers → LayerAggregator (now reads active system policy weights) → safety overrides → timing → base response.
3. Profile Applicator checks Redis cache for user profile.
4. If profile exists AND 20+ predictions: adjust confidence using accuracy_by_setup match; shift timing based on timing_profile; attach matching personalized warnings from active_warnings.
5. If no profile: return base response unchanged (identical to today).
6. Async log: write prediction + policy_version + profile_version to user_predictions (non-blocking).
7. Return response to client.

Latency impact: Profile read from Redis: ~1–2ms. Adjustment computation: simple arithmetic. Total added latency: <3ms on a 500–2000ms response.

8.2 Outcome Collection (Every 4 Hours)

Celery Beat task queries user_predictions for entries without outcomes, fetches current prices via DataFetcher, computes direction_correct at each timeframe, writes to prediction_outcomes.

8.3 Per-User Profile Builder (Weekly)

1. For each user with new outcome data since last update:
2. SQL aggregation: accuracy rates by (token_type, market_condition, signal), hold patterns, request frequency analysis, outcome streaks.
3. Compact stat summary sent to LLM in structured prompt.
4. LLM returns JSON: behavioral traits, confidence multipliers, timing classification, mistake patterns, personalized warning templates.
5. Validate response (bounds check all values), version, and write to user_profiles. Archive previous version.
6. Invalidate Redis cache for this user.

8.4 System-Wide Optimizer (Weekly)

1. SQL aggregation across ALL users: per-layer accuracy, conflict outcomes, threshold performance.
2. Summary sent to LLM. Returns proposed policy with weight/threshold changes.
3. Policy enters shadow mode. After 500+ shadow predictions with outcomes, metrics compared.
4. Promote if better, discard if worse. Rolling monitor auto-rollbacks degraded active policies.